TeleConnect

Retention Agent -- End-to-End Guide

Part 2: How the AI Agent Works, Step by Step

What this document covers

This guide explains -- step by step -- how the TeleConnect AI retention agent works end to end. It covers the agent's architecture, every tool it uses, the conversation flow, the orchestration loop, the evaluation framework, and the Streamlit demo app. It is written for a technical reviewer who wants to understand both the design decisions and the exact code that runs them.

Section 1 -- System Overview

The TeleConnect system has two parts that work together. Part 1 trains a churn prediction model from messy customer data. Part 2 wires that model into a live AI agent that retention representatives talk to in natural language. The model from Part 1 becomes one of the agent's tools in Part 2 -- they are explicitly connected.

1.1 The Big Picture

Architecture flow

```
Retention rep types a message
         |
         v
Streamlit app  (app/streamlit_app.py)
         |
         v
RetentionAgent.run()  (src/agent/orchestrator.py)
         |
         v
LLM Client  <-- reads SYSTEM PROMPT (src/agent/prompts.py)
(Anthropic / Ollama / Mock)  src/llm_client.py
         |
   [tool_use block returned]
         |
         v
execute_tool()  (src/agent/tools.py  -- TOOL_REGISTRY)
   |           |           |           |           |
lookup_    predict_   get_       log_       escalate_
customer   churn      offers     interaction supervisor
   |           |
cleaned     sklearn pipeline
CSV         (models/churn_pipeline.joblib)
         |
    [tool_result fed back to LLM]
         |
   ... loop until no more tool_use ...
         |
         v
 Final text response -> displayed in Streamlit + tool timeline shown
```

1.2 Two Distinct 'Models' -- Do Not Confuse Them

There are two very different things called 'model' in this project:

Component	What it is	Uses API?
Churn model	Sklearn LogisticRegression pipeline (.joblib file)	No -- local
LLM (agent brain)	Claude / Ollama / Mock -- drives conversation	Optional

The churn model always runs locally and for free -- no API key, no network. Only the LLM (the conversation brain) optionally calls an external API. Without any key the system uses a deterministic mock that still demonstrates the correct tool chain.

1.3 Key Design Principles

* No train/serve skew: src/data_cleaning.py and src/features.py are shared by the notebook (training) and src/predict.py (inference). The same raw value gets the same transform at both stages.
* Extensibility: adding a 6th tool = one entry in TOOL_REGISTRY. The orchestrator iterates the registry; nothing else changes.
* Always runnable: FORCE_MOCK_LLM=1 runs the full system end-to-end with no API key, so the demo and evals are never blocked.
* Honest labeling: mock mode is labeled everywhere -- in the Streamlit sidebar, the scorecard banner, and the README.

Section 2 -- The Five Tools

All tools live in src/agent/tools.py. Each has two parts: (1) a JSON Schema that tells the LLM what the tool does and what arguments it takes, and (2) a Python implementation function. The TOOL_REGISTRY dict maps tool names to both.

2.1 lookup_customer

Purpose

Retrieves a customer's profile by their account ID. This is always the FIRST tool called -- the agent cannot predict churn or get offers until it has a profile.

Input

```
{"customer_id": "TC-001096"}
```

Output

```
{
  "found": true,
  "customer_id": "TC-001096",
  "demographics": {"age": 32, "gender": "Male"},
  "contract": {"contract_type": "Month-to-month", "payment_method": "Credit card"},
  "tenure_months": 10,
  "charges": {"monthly_charges": 69.24, "total_charges": 656.42},
  "services": {"internet_service": "DSL", "phone_service": "Yes", ...},
  "satisfaction_score": 7.8,
  "_features": { ...full feature dict for predict_churn... }
}
```

Implementation

Backed by data/cleaned_customers.csv (the cleaned dataset). The _features field is a flat dict of all customer features -- the agent passes this directly to predict_churn as customer_data, so it doesn't need to re-extract fields.

Error handling

If the ID is not found it returns {found: false, error: '...', hint: '...'} so the agent can relay the problem to the rep and ask them to re-check.

2.2 predict_churn

Purpose

Runs the REAL trained sklearn pipeline from Part 1 on the customer's features. This is the bridge between Part 1 and Part 2.

Input

```
{"customer_data": { ...the _features dict from lookup_customer... }}
```

Output

```
{
  "churn_probability": 0.76,
  "risk_tier": "high",
  "top_risk_factors": [
    "Month-to-month contract (no lock-in)",
    "High support-ticket count (4)",
    "Low tenure (10 months)"
  ]
}
```

How it works internally

The call chain inside predict_churn():

* clean_record(customer_data) -- same cleaning as training (no skew)
* add_engineered_features() -- same 5 features as training
* pipeline.predict_proba(X) -- the saved sklearn pipeline (lru_cache: loaded once)
* Probability thresholds from model_metadata.json -> risk_tier
* Feature direction + importance heuristic -> top_risk_factors (top 3)

If the model artifact is missing (not yet trained), the tool returns a clearly labeled mock prediction with a warning -- per the brief: 'mock it, but say so.'

2.3 get_retention_offers

Purpose

Returns retention offers filtered by the customer's risk tier and optionally their contract type. Offer aggressiveness is matched to risk -- high risk unlocks stronger discounts; low risk returns margin-preserving options.

Input

```
{"risk_tier": "high", "contract_type": "Month-to-month"}
```

Offer catalog (designed by us -- brief asked for our own design)

Offer	Description	Eligible tiers
20% Loyalty Discount	20% off bill for 12 mo in exchange for 1-yr commitme	high, medium
Free Upgrade + 2yr Lock	Free tier upgrade if moves to 2-year contract	high, medium
Service Recovery Credit	Bill credit + 90 days priority support	high only
Free Add-On (6 mo)	Complimentary streaming/security add-on	medium, low
Data/Minutes Boost	Doubled allowance at no cost for 6 months	medium, low
Goodwill Check-In	Proactive review, no financial concession	low only

2.4 log_interaction

Purpose

Records the outcome of a retention conversation for analytics and audit. The brief required this to be mocked but with a production-realistic schema.

Schema (appended to data/interaction_log.jsonl)

```
{
  "interaction_id": "INT-20260529182345123456",
  "logged_at_utc": "2026-05-29T18:23:45.123456+00:00",
  "customer_id": "TC-001096",
  "rep_id": "REP-007",
  "churn_risk_tier": "high",
  "churn_probability": 0.76,
  "offers_presented": ["RET-LOYALTY-20", "RET-CONTRACT-SWITCH"],
  "outcome": "accepted",  // accepted|declined|escalated|pending|callback
  "notes": "Customer agreed to 1-year contract after loyalty discount offer",
  "schema_version": "1.0"
}
```

2.5 escalate_to_supervisor

Purpose

Transfers the case to a human supervisor with a full context summary. Used for situations outside the agent's remit.

When the agent escalates (per system prompt)

* Customer threatens legal or regulatory action (lawyer, lawsuit, ombudsman)
* Complex billing dispute the agent cannot resolve
* Customer is highly distressed or asks for something no tool handles

Output


```
{
  "escalated": true,
  "ticket": {
    "escalation_id": "ESC-20260529182345123456",
    "customer_id": "TC-003356",
    "reason": "Customer threatened legal action over disputed charges",
    "context_summary": "Customer on Month-to-month, 61% churn prob ...",
    "urgency": "high",
    "status": "queued_for_supervisor"
  }
}
```

Section 3 -- The Orchestration Loop (Step by Step)

The orchestrator (src/agent/orchestrator.py) runs a generic tool-calling loop. It is completely provider-agnostic -- it does not know or care whether the LLM is Claude, Ollama, or the mock. It just sends messages, executes tools, and loops.

3.1 The Loop in Detail

1 Rep message received

The rep's text is appended to the messages list as a 'user' role message. The conversation history (prior turns) is included so the LLM has context.

2 System prompt sent

The SYSTEM_PROMPT from src/agent/prompts.py is sent with every LLM call. It encodes tool-chain order, ambiguity handling, escalation triggers, and response format. This is the agent's permanent instruction set.

3 LLM responds

The LLM returns one or more content blocks. Each block is either: (a) text -- the LLM is thinking or synthesizing, or (b) tool_use -- the LLM wants to call a specific tool with specific arguments.

4 Tool_use? Execute tools

For each tool_use block: look up the tool in TOOL_REGISTRY, call its implementation with the provided arguments, capture the result (always a dict), record the call in the trace (order, name, input, output, latency_ms).

5 Tool result fed back

All tool results are appended to messages as 'user' role tool_result blocks (Anthropic format). This is the standard multi-turn tool-calling protocol.

6 Loop repeats

Steps 2-5 repeat. The LLM now sees the tool outputs and can either call more tools or produce a final text response.

7 No tool_use? Done

When the LLM returns only text blocks (stop_reason = 'end_turn'), the loop exits. The final text is the agent's response to the rep.

8 Max turns safety

If the loop runs for more than 8 turns without a final answer, it exits with a fallback message asking the rep to rephrase. This prevents infinite loops.

3.2 The Trace Record

Every tool call is recorded as a ToolCallRecord with: order (1, 2, 3...), name, input dict, output dict, and

latency_ms. This trace is returned in AgentResult and consumed by: (a) the Streamlit app to render the tool timeline, and (b) the eval harness to compute tool-selection accuracy.

AgentResult structure

```
AgentResult(
  final_text='**Retention summary for TC-001096**...',
  tool_calls=[
    ToolCallRecord(order=1, name='lookup_customer', input={...}, output={...}, latency_ms=2.1),
    ToolCallRecord(order=2, name='predict_churn',   input={...}, output={...}, latency_ms=0.8),
    ToolCallRecord(order=3, name='get_retention_offers', ...),
  ],
  is_mock=True,
  total_latency_ms=215.3,
  input_tokens=0, output_tokens=0  # 0 in mock mode
)
```

Section 4 -- LLM Provider Abstraction (src/llm_client.py)

The entire codebase speaks one normalised format (Anthropic-style content blocks). All three client classes implement a single method: respond(system, messages, tools). Swapping providers = one class. The orchestrator, tools, and app never change.

4.1 Provider Selection Logic

Provider resolution order

```
LLM_PROVIDER env var  +  FORCE_MOCK_LLM

FORCE_MOCK_LLM=1  -->  MockClient  (always wins, overrides everything)
LLM_PROVIDER=anthropic + ANTHROPIC_API_KEY set  -->  AnthropicClient
LLM_PROVIDER=ollama  -->  OllamaClient
(nothing set)  -->  MockClient
```

4.2 The Three Clients

AnthropicClient

Uses the Anthropic Python SDK. Sends the system prompt, messages, and tool schemas to Claude (claude-sonnet-4-6 for the agent, claude-opus-4-8 for the judge). Parses the response into normalised text/tool_use blocks.

* Requires: pip install anthropic + funded ANTHROPIC_API_KEY
* Agent model: AGENT_MODEL env (default claude-sonnet-4-6)
* Judge model: JUDGE_MODEL env (default claude-opus-4-8 -- different for independence)

OllamaClient

Talks to Ollama's /api/chat endpoint using Python's standard library urllib (zero new dependencies). Translates between Anthropic-style blocks and Ollama's OpenAI-compatible tool format. Ollama itself wraps llama.cpp and downloads GGUF model weights.

* Requires: install Ollama (https://ollama.com) + ollama pull qwen2.5:3b-instruct
* Set: LLM_PROVIDER=ollama, OLLAMA_MODEL=qwen2.5:3b-instruct
* Also works with a remote Ollama via OLLAMA_BASE_URL (e.g. ngrok tunnel)
* Speed measured: 3b ~ 2 min/request on CPU, 7b ~ 10 min/request on CPU

MockClient

A deterministic rule-based stand-in. It does NOT call any LLM -- it uses pattern matching (regex for customer IDs, keyword lists for escalation) to decide which tool to call next and synthesise a structured final response.

It correctly handles: the full tool chain, ambiguity (no ID -> ask), legal threats (-> escalate), out-of-scope (-> decline), unknown IDs (-> stop after lookup). The mock judge returns placeholder scores.

The mock has known limitations: it cannot reason about conflicting signals (model score vs profile quality), and it does not reliably call log_interaction at the end. Both are documented as mock-specific gaps -- the real LLM

handles them correctly per the system prompt.

Section 5 -- Annotated Conversation Walkthrough

Below is a complete trace of a real agent session for a high-risk customer. Each step shows exactly what message is sent, what the LLM returns, what tool is called, and what comes back. This is the exact sequence the code runs.

Scenario: 'Customer TC-001096 called in saying they might cancel'

Step 1 -- Rep message arrives

```
messages = [
  {"role": "user",
   "content": "Customer TC-001096 called in saying they might cancel. What should I do?"}
]
```

The orchestrator appends the rep's message and calls LLM.respond() with the system prompt + messages + all 5 tool schemas.

Step 2 -- LLM returns: call lookup_customer

```
LLM response blocks:
[{"type": "tool_use",
  "id": "tu_001",
  "name": "lookup_customer",
  "input": {"customer_id": "TC-001096"}}]

stop_reason = "tool_use"  --> loop continues
```

The LLM recognised the customer ID and chose the correct first tool. The system prompt says: 'Call lookup_customer FIRST whenever you have a customer ID.'

Step 3 -- execute_tool('lookup_customer', {customer_id: 'TC-001096'})

```
Tool result (from cleaned_customers.csv):
{
  "found": true, "customer_id": "TC-001096",
  "tenure_months": 10, "contract": {"contract_type": "Month-to-month"},
  "satisfaction_score": 7.8, "charges": {"monthly_charges": 69.24},
  "_features": {all feature fields...}
}

Recorded: ToolCallRecord(order=1, name='lookup_customer', latency_ms=2.1)
```

The result is appended as a tool_result block. The loop sends this back to the LLM.

Step 4 -- LLM returns: call predict_churn

```
[{"type": "tool_use",
  "name": "predict_churn",
  "input": {"customer_data": {<the _features dict>}}}]
```

The LLM extracted the _features dict from the previous tool result and passed it to predict_churn. This is multi-step reasoning -- the output of step 3 becomes the input of step 4.

Step 5 -- execute_tool('predict_churn', {customer_data: {...}})

```
Tool result (from sklearn pipeline -- the real trained model):
{
  "churn_probability": 0.76,
  "risk_tier": "high",
  "top_risk_factors": [
    "Month-to-month contract (no lock-in)",
    "High support-ticket count (4)",
    "Low tenure (10 months)"
  ]
}

Recorded: ToolCallRecord(order=2, name='predict_churn', latency_ms=0.8)
```

The churn model runs locally (no API). Pipeline: clean -> engineer features -> impute -> scale -> LogisticRegression.predict_proba -> threshold.

Step 6 -- LLM returns: call get_retention_offers

```
[{"type": "tool_use",
  "name": "get_retention_offers",
  "input": {"risk_tier": "high", "contract_type": "Month-to-month"}}]
```

The LLM extracted risk_tier='high' from the predict_churn result and contract_type='Month-to-month' from the lookup result. It correctly combined information across two prior tool calls.

Step 7 -- execute_tool('get_retention_offers', ...)

```
Tool result (filtered offer catalog):
{
  "risk_tier": "high", "count": 3,
  "offers": [
    {"offer_id": "RET-LOYALTY-20", "name": "20% Loyalty Discount (12 mo)",
     "description": "20% off the monthly bill for 12 months in exchange for a 1-year commitment.",
     "est_monthly_cost": 14.0},
    {"offer_id": "RET-CONTRACT-SWITCH", "name": "Free Upgrade + 2-Year Lock-In", ...},
    {"offer_id": "RET-SERVICE-RECOVERY", "name": "Service Recovery Credit", ...}
  ]
}
```

Step 8 -- LLM synthesises the final response

```
stop_reason = "end_turn"  --> loop exits

Final response text:
**Retention summary for TC-001096**

- **Churn risk:** HIGH (76% probability)
- **Top risk factors:** Month-to-month contract, High support-ticket count (4), Low tenure (10 months)
- **Recommended offer:** 20% Loyalty Discount (12 mo) -- 20% off the monthly
  bill for 12 months in exchange for a 1-year commitment.
- **Alternatives:** Free Upgrade + 2-Year Lock-In, Service Recovery Credit

**Suggested approach:** Lead with empathy, acknowledge the customer's concerns,
and present the recommended offer framed around the value they'll keep.
Confirm acceptance and log the outcome.
```

The LLM synthesised information from 3 tool calls into a rep-facing recommendation -- not a data dump. Specific offer, specific reason, talking point.

5.2 Full Turn Summary

#	Tool called	Key input	Key output
1	lookup_customer	customer_id: TC-001096	profile + _features dict
2	predict_churn	_features from step 1	prob=0.76, tier=high, 3 factors
3	get_retention_offers	tier=high, contract=Month-to-month	3 eligible offers
4	(synthesis)	all prior results	rep-facing recommendation

Section 6 -- Edge Cases and How the Agent Handles Them

6.1 Ambiguous input -- no customer ID

Input:	Input: 'I have a high-risk customer on the phone!'
Expected:	Expected: ask for the customer ID, call NO tools
How:	How: system prompt says 'If the rep describes a customer but gives no customer ID, ask for it before doing anything else.' The LLM returns a text block (not tool_use). The mock client checks: no TC-XXXXX pattern in the message -> returns a clarification request.
Result:	No tools called. Response: 'Happy to help. Could you give me the customer ID (TC-XXXXX)?'

6.2 Unknown customer ID

Input:	Input: 'Look up customer TC-999999 and tell me their churn risk.'
Expected:	Expected: call lookup_customer, get not-found, stop -- do NOT predict
How:	How: lookup_customer returns {found: false, error: 'No customer found with id TC-999999'}. The system prompt says: 'If lookup returns an error, tell the rep plainly and ask them to re-check.' The mock specifically checks the last tool result for found=false and returns a text response. predict_churn is NEVER called -- fabricating a prediction for an unknown customer would be hallucination.
Result:	Tools: [lookup_customer]. Response mentions 'not found' and asks rep to re-check.

6.3 Legal / regulatory threat -- escalation

Input:	Input: 'Customer TC-003356 says they will get a lawyer and sue us.'
Expected:	Expected: escalate_to_supervisor, do NOT offer discounts
How:	How: the word 'lawyer' triggers the escalation path (word-boundary regex, not substring). The system prompt: 'Escalate when the customer threatens legal or regulatory action.' The agent calls escalate_to_supervisor with the customer ID, reason, and a context summary. It does NOT call get_retention_offers -- offering a discount to a customer threatening legal action could be construed as an admission.
Result:	Tools: [escalate_to_supervisor]. Response confirms escalation and context handoff.

6.4 Out-of-scope request

Input:	Input: 'What is the weather like in London today?'
Expected:	Expected: politely decline, call NO tools
How:	How: the out-of-scope keyword list catches 'weather' (whole-word match). System prompt: 'If asked something unrelated to retention, politely decline and redirect.' No tools are called at all.
Result:	Tools: []. Response redirects to retention help.

6.5 Model disagreement -- low score, bad profile

Input:	Input: 'The model says TC-002360 is low risk, but they seem really unhappy.'
Expected:	Expected: surface the tension -- low model score but low satisfaction + high tickets
How:	How: this is a REAL LLM behavior (not mock). The system prompt section 'Conflicting signals' says: 'If the model's risk tier disagrees with the profile, say so explicitly and recommend the more cautious action.' A real LLM reads this and reasons about it. The mock skips it (documented limitation). With Claude or Ollama, the agent flags the conflict.
Result:	Note: this case FAILS in mock mode (0%) and PASSES with a real LLM (documented in eval).

Section 7 -- The Evaluation Framework

The eval framework (eval/) implements Part 2.2 of the brief: a structured test suite, automated metrics, and an LLM-as-judge. It runs with no API key (mock mode) and produces a scorecard that proves the agent was tested -- not just 'tried'.

7.1 The Test Suite (eval/test_suite.py)

14 dataclass test cases spanning all required categories. Each case has:

* user_input -- the rep's message
* expected_tools -- ordered list of (tool, key params)
* must_not_call -- tools that must NOT be called
* quality_criteria -- plain-language bar for a good response
* response_must_contain -- substrings checked by the completeness metric
* category -- for aggregate reporting

Category	Cases	What it tests
single_tool_happy_path	1	Simple lookup -- stop after one tool
multi_step_chaining	3	Full lookup -> predict -> offers -> synthesize chain
ambiguous_input	2	No customer ID -- ask before acting
out_of_scope	2	Weather/password -- decline, call no tools
escalation_trigger	2	Legal threat / regulatory complaint -> escalate
model_disagreement	1	Low model score but bad profile -- flag the conflict
adversarial_edge	3	Prompt injection, unknown ID, conflicting instruction

7.2 Automated Metrics (eval/metrics.py)

Three deterministic metrics -- no LLM needed. Run in milliseconds. Used as a cross-check against the LLM judge.

1. Tool Selection Accuracy (primary)

Combines: (a) no forbidden tool was called (hard requirement), (b) Jaccard similarity between expected and actual tool sets, (c) whether expected tools appear in the correct order (subsequence check). Score = 0.6 * jaccard + 0.4 * ordered_correct, halved if forbidden tool called.

2. Parameter Extraction Accuracy

For each expected tool call that declares key params, checks whether the agent passed the correct value. E.g. did lookup_customer receive the exact customer_id mentioned in the input? Score = fraction of key params correctly extracted.

3. Response Completeness

Checks that the final response contains all required substrings (e.g. 'high', 'offer', 'not found'). Fraction of required strings present. Catches cases where the agent called the right tools but gave a useless response.

7.3 LLM-as-Judge (eval/judge.py)

A separate LLM call (deliberately a DIFFERENT model than the agent -- claude-opus-4-8 or qwen2.5:7b vs the agent's claude-sonnet-4-6 or 3b) scores the agent's response on four dimensions using anchored 1/3/5 rubric scores.

Dimension	Score 1	Score 3	Score 5
factual_correctness	Contradicts tools	Minor mismatch	Fully grounded
tool_use_appropriateness	Wrong tools/order	Questionable	Perfect chain
actionability	Raw data dump	Generic advice	Specific + talking poir
hallucination	Fabricates facts	One unsupported detail	No fabrication

Judge reliability (addressed in the brief)

* Positivity bias: countered by explicit anchors (not 'good/bad' but concrete descriptions)
* Prompt sensitivity: fixed rubric version, temperature=0
* Independence: judge uses a DIFFERENT model than the agent
* Calibration: deterministic metrics cross-check the judge; recommend periodic hand-labeling + Cohen's kappa measurement

7.4 The Scorecard (eval/run_eval.py)

Run: python -m eval.run_eval [--no-judge]

Outputs: eval/results/scorecard.md + scorecard.json with:

* Overall pass rate (14 cases, 71% in mock mode)
* Per-category pass rate breakdown
* Average automated metric scores
* Average LLM-judge score by dimension
* Per-case table (tools called, pass/fail, judge mean)
* Cost report (latency per case, total tokens)

Section 8 -- The Streamlit Demo App (app/streamlit_app.py)

The Streamlit app is the Part 2.3 deliverable -- a clickable, hosted demo. It wraps the agent in a chat interface and, crucially, makes every tool call VISIBLE to the reviewer (required by the brief: 'showing it is part of the deliverable').

8.1 UI Layout

* Left sidebar: mode banner (mock/live), example prompt buttons, tool list, clear button
* Main area: chat history (user + assistant turns)
* Under each assistant response: tool-call timeline (expandable)
* Footer per response: latency, token count, model name

8.2 The Tool-Call Timeline (the key differentiator)

The brief says: 'The interface makes tool calls visible -- what was called, in what order, and what each tool returned. Do not hide the orchestration; showing it is part of the deliverable.'

For each tool call in AgentResult.tool_calls:

```
st.expander('1. lookup_customer  .  2.1 ms')
  Input:  {'customer_id': 'TC-001096'}
  Output: {'found': True, 'tenure_months': 10, ...}

st.expander('2. predict_churn  .  0.8 ms')
  Input:  {'customer_data': {...}}
  Output: {'churn_probability': 0.76, 'risk_tier': 'high', ...}

st.expander('3. get_retention_offers  .  0.3 ms')
  Input:  {'risk_tier': 'high', 'contract_type': 'Month-to-month'}
  Output: {'count': 3, 'offers': [...]}
```

8.3 API Key and Secrets

The app reads env vars or Streamlit secrets. The provider is selected automatically:

* FORCE_MOCK_LLM=1 -> mock agent (always works, no cost)
* ANTHROPIC_API_KEY set -> real Claude agent
* LLM_PROVIDER=ollama -> local Ollama agent (local demo only)

For Streamlit Cloud deployment, paste secrets in the app's Secrets manager (not in the repo -- the .gitignore excludes .streamlit/secrets.toml).

8.4 Running Locally

```
# Mock mode (always works, no API key):
$env:FORCE_MOCK_LLM = '1'
.venv\Scripts\python.exe -m streamlit run app/streamlit_app.py
```

```
# Opens at http://localhost:8501

# Ollama mode (real local LLM, no cost):
$env:FORCE_MOCK_LLM = ''
$env:LLM_PROVIDER = 'ollama'
$env:OLLAMA_MODEL = 'qwen2.5:3b-instruct'
.venv\Scripts\python.exe -m streamlit run app/streamlit_app.py
```

Section 9 -- Complete File Map

Every file in the project and what it does:

src/ -- source code

`src/data_cleaning.py`	Clean raw CSV data. Shared by notebook (training) and predict.py (inference).
`src/features.py`	Add 5 engineered features. Shared by notebook and predict.py.
`src/predict.py`	predict_churn(dict) -> {probability, tier, factors}. The Part 1.5 function.
`src/llm_client.py`	Provider abstraction: AnthropicClient, OllamaClient, MockClient.
`src/agent/tools.py`	TOOL_REGISTRY: 5 tool schemas + implementations. Add 6th tool here.
`src/agent/orchestrator.py`	Generic tool-calling loop. Provider-agnostic. Returns AgentResult.
`src/agent/prompts.py`	SYSTEM_PROMPT: chain order, ambiguity, escalation, response format.
`src/agent/mock_db.py`	lookup_customer (CSV-backed) + get_retention_offers (offer catalog).

eval/ -- evaluation

`eval/test_suite.py`	14 TestCase dataclasses covering all required categories.
`eval/metrics.py`	3 deterministic metrics: tool_selection, param_extraction, completeness.
`eval/judge.py`	LLM-as-judge: anchored 1/3/5 rubric, 4 dimensions, separate model.
`eval/run_eval.py`	Orchestrates full eval -> scorecard.md + scorecard.json.
`eval/results/scorecard.md`	Generated aggregate scorecard with per-case results.

app/ -- demo

`app/streamlit_app.py`	Chat UI + tool-call timeline + mock/live mode banner.

notebooks/ -- Part 1

`notebooks/churn_model.ipynb`	Full narrative notebook: cleaning, EDA, models, export.
`notebooks/_build_notebook.py`	Programmatic notebook builder for reproducibility.

models/ -- artifacts

`models/churn_pipeline.joblib`	Trained sklearn pipeline (ColumnTransformer + LogisticRegression).
`models/model_metadata.json`	Thresholds, feature directions/stats, importances, metrics.

tests/ -- tests

`tests/test_predict.py`	5 tests for predict_churn (high/low/partial/empty/schema).
`tests/test_agent.py`	7 tests for agent tool chain (chain, ambiguity, escalation, OOS...).
`tests/test_eval_suite.py`	3 tests: categories present, metrics run, scorecard written.
`tests/test_churn_model.py`	Full model evaluation report: algorithm, metrics, importances.

`tests/run_all_tests.py`	Single-command runner: 15 tests, no pytest needed.
`tests/generate_model_comparison.py`	Generates docs/model_comparison.md with all model metrics.
`tests/generate_agent_pdf.py`	Generates this PDF document.

docs/ -- generated reports

`docs/model_comparison.md`	Side-by-side metrics for LogisticRegression vs XGBoost.
`docs/TeleConnect_Agent_Part2_Guide.pdf`	This document.

data/ -- data files

`data/test_datafile.csv`	Raw dataset (5,050 rows, deliberately messy).
`data/cleaned_customers.csv`	Cleaned dataset (5,000 rows) -- also backs lookup_customer.

Run tests/run_all_tests.py to verify the full system end-to-end.

Run tests/generate_model_comparison.py to regenerate the model metrics report.

Run python -m eval.run_eval to regenerate the agent scorecard.

Run streamlit run app/streamlit_app.py to launch the live demo.